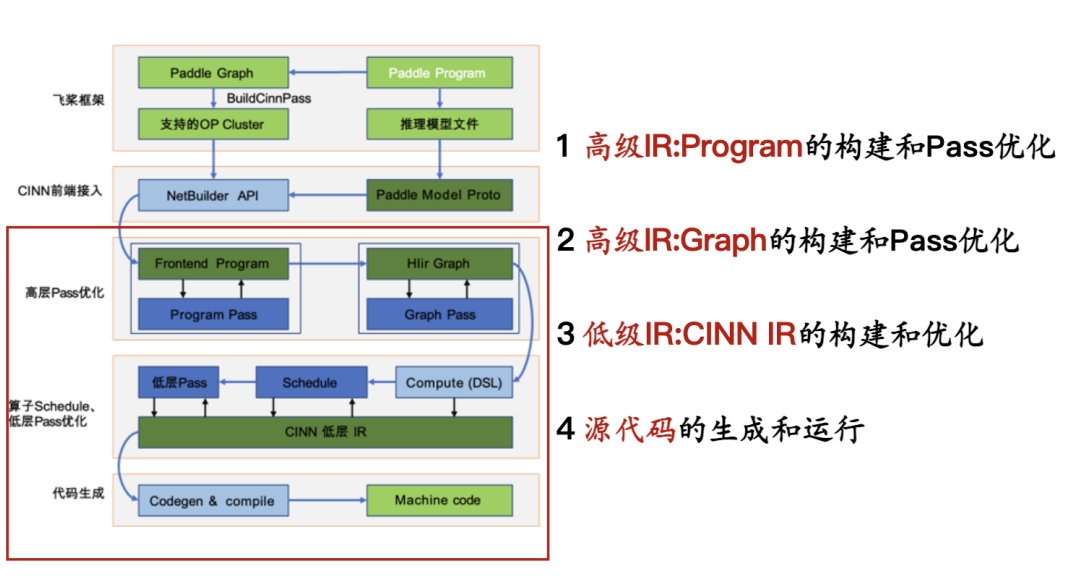


CINN学习梳理

执行路径梳理



入口函数： build_and_get_output

</> pybind端： cinn/pybind/frontend.cc

C++ | 收起 ^

```
1 // pybind 绑定, python 端获取计算结果
2 .def(
3     "build_and_get_output",
4     [](Program &self,
5         const common::Target &target,
6         const std::vector<Variable> &tensor_inputs,
7         const std::vector<py::array> &input_data,
8         const std::vector<Variable> &tensor_outputs,
9         const std::vector<std::string> &passes =
10         std::vector<std::string>{},
11         std::shared_ptr<hlir::framework::Scope> scope = nullptr) { // None
12     // 【1. Apply 前端: program 和 graph pass】
13     graph = Optimize(&self, fetch_ids, target);
14     // 【2. 构造 GraphCompiler, 基于 graph 做编译】
15     scope = hlir::framework::BuildScope(target, graph, scope);
16     hlir::framework::GraphCompiler gc(target, scope, graph);
17     hlir::framework::GraphCompiler::CompileOptions options;
```

```

18     options.with_instantiate_variables = true;
19     auto gc_fetch_ids                = fetch_ids;
20     // 【【3. Build 出 runtime_program, 构造低层 IR】】
21     // 【【每个 op Compute + LowerFunc + Schedule 三步走 】】
22     const auto &result                = gc.Build(options,
std::move(gc_fetch_ids));
23     const auto &program                = result.runtime_program;
24
25     // 【【4. 按顺序执行 Instruction】】
26     program->Execute();
27     return outputs;
28 }

```

1. Apply 前端：program 和 graph pass

数据结构：frontend::Program、frontend::Variable、hlir::framework::Graph、

Program --> [Program pass] --> **Program** --> [convert to Graph] --> **Graph** --> [Graph pass] --> **Graph**

Program pass:

AutoCast, Decomposer, **Removeldentity**, CastCollapsing, TransposeCollapsing, **Removeldentity**,

[TransposeFoldingInput, GemmRewriter, TransposeFoldingOutput, GemmRewriter],

FillConstantRewriter, FillConstantFolding, **Removeldentity**, DeadCodeEliminate

Graph Pass:

ConstantFolding, [DenseMergePass], [TransToCustomCallPass],

[CommonSubexpressionEliminationPass], [ReduceSplit],

[OpFusionPass, FusionMergePass], [BuildNonFusedGroupsPass]

SingleGroupOptimizePass, [CheckFusionAccuracyPass, TransToCustomCallPass]

</> cinn/frontend/optimize.cc

C++ | 收起 ^

```

1 std::shared_ptr<hlir::framework::Graph> Optimize(frontend::Program* program,
2                                                  const
std::unordered_set<std::string>& fetch_ids,
3                                                  common::Target target,
4                                                  const OptimizeOptions&
options) {
5     cinn::hlir::framework::PassPrinter::GetInstance()->Begin(fetch_ids);
6     frontend::ProgramPass::Apply(program, fetch_ids, target,
options.program_passes);

```

```

7
8  auto graph = std::make_shared<hlir::framework::Graph>(*program, fetch_ids,
    target);
9
10 hlir::framework::ApplyPasses(graph.get(), options.graph_passes);
11 cinn::hlir::framework::PassPrinter::GetInstance()->End();
12 return graph;
13 }

```

Program --> Graph 的转换:

序列型Program并不适合分析拓扑结构和做一些Fusion优化，因此使用Graph结构

</> Graph 构造函数: cinn/hlir/framework/graph.cc

C++ | 收起 ^

```

1 void Graph::Initialize(const frontend::Program& prog,
2                        const std::unordered_set<std::string>& fetch_var_ids,
3                        const Target& target) {
4     // 遍历 prog 中的每个 op
5     for (size_t i = 0; i < prog.size(); i++) {
6         // 遍历 op 的输入和输出，注册 NodeData
7         // 注册当前 op 为 Node
8         this->RegisterNode(node_tmp->id(), node_tmp);
9     }
10    // graph 的 attr 中保存 input 和 output 的 shape 和 dtype
11    // 【注】：实际上 infershape 发生在 NetBuilder 添加 op 的时候
12    this->attrs["infershape"] = std::make_shared<absl::any>(shape_dict);
13    this->attrs["inferdtype"] = std::make_shared<absl::any>(dtype_dict);
14 }

```

2. 构造 Scope 和 GraphCompiler

数据结构:

```

using hlir::framework::Variable = absl::variant<Tensor>; hlir::framework::Tensor
hlir::framework::GraphCompiler

```

2.1 scope = hlir::framework::BuildScope(target, graph, scope);

scope 只有一个成员变量，用来保存 name 到 Variable (Tensor) 的映射，BuildScope 往 scope 中添加 Variable

```
absl::flat_hash_map<std::string, std::unique_ptr<Variable>> data_;
```

2.2 构造 GraphCompiler

传入参数：target、scope（保存 Variable）、graph（图拓扑结构）

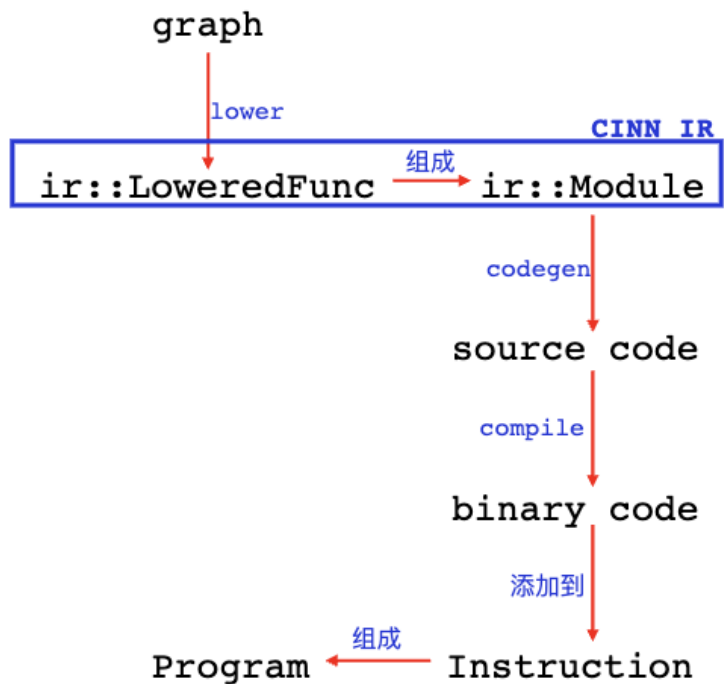
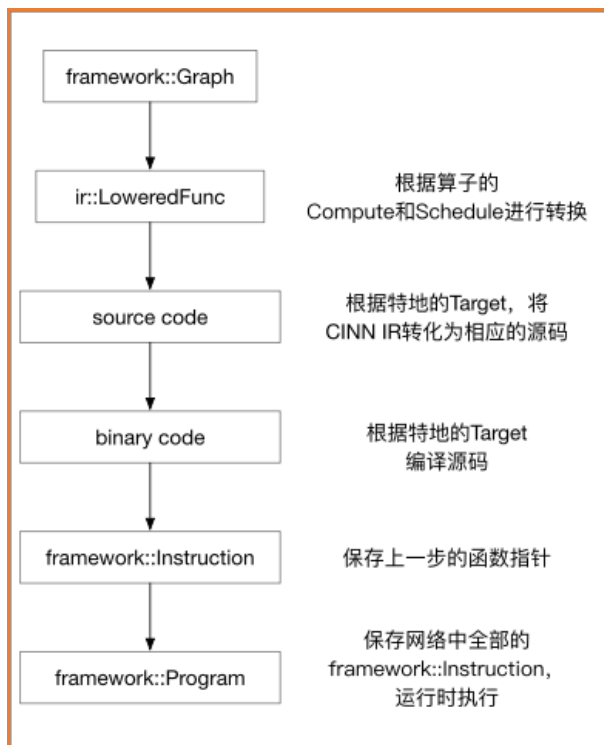
hlir::framework::GraphCompiler gc(target, scope, graph);

</> GraphCompiler

C++ | 收起 ^

```
1 class GraphCompiler final {
2     // 有很多的成员变量
3     private:
4         // parallel compiler
5         std::shared_ptr<ParallelCompiler> parallel_compiler_;
6         // 构造函数初始化的三个成员变量
7         Target target_;
8         std::shared_ptr<Graph> graph_;
9         std::shared_ptr<Scope> scope_;
10        // mapping a function's name to its input arguments' names
11        std::map<std::string, std::vector<std::string>> function2input_args_;
12        // mapping a function's name to its output arguments' names
13        std::map<std::string, std::vector<std::string>> function2output_args_;
14        // fetch var ids in cinn and the corresponding var nodes will not be fused
        so as to get the result
15        std::unordered_set<std::string> fetch_var_ids_;
16
17        absl::flat_hash_map<std::string, std::string> prefix2full_namemap_;
18        // map dst reuse var to the src var sharing buffer
19        absl::flat_hash_map<std::string, std::string> reuse_vars_map_;
20
21        std::unique_ptr<backends::Compiler> compiler_;
22        CompileOptions compile_options_;
23
24        ir::Module::Builder m_builder_;
25 };
```

3. Build 出 runtime_program, 构造低层 IR



首先从graph出发，Lower到 CINN IR，这里面有Schedule的过程。然后codegen 出 source code并编译出可执行的二进制代码，将地址添加到hlir::Instruction中，而hlir::Instruction组成了hlir::Program。

```
</> cinn/hlir/framework/graph_compiler.cc
```

C++ | 收起 ^

```

1 GraphCompiler::CompilationResult GraphCompiler::Build(const
  GraphCompiler::CompileOptions& options,
2
  std::unordered_set<std::string>&& fetch_var_ids,
3                                     void* stream) {
4   // 并行编译【略】
5   if (FLAGS_cinn_parallel_compile_size) { }
6   // 【【【串行编译】】】
7   std::vector<std::vector<Node*>> groups;
8   // 【初始化】如果没有 group, 会把每个 op node 作为一个 group, 每个 group 对应一个
  Instruction
9   if (options.groups.empty() && graph_>groups.empty() && graph_-
    >fusion_groups.empty()) { }
10  // 【遍历 group】开始 LoweredFunc, 如果 options.lowered_funcs 为空, 需要根据
    group 生成 LoweredFunc
11  if (options.lowered_funcs.empty()) {
12    // 如果 graph 已经分出来了 fusion_groups, 那么对 fusion_groups 中的每个 group
    做 OpLowering
13    if (!graph_>fusion_groups.empty()) {
14      auto& dtype_dict = graph_-
    >GetMutableAttrs<absl::flat_hash_map<std::string, Type>>("inferdtype");
15      auto& shape_dict = graph_-
    >GetMutableAttrs<absl::flat_hash_map<std::string, shape_t>>("inferredshape");
  
```

```

16
17     OpLowerer op_lowerer(dtype_dict, shape_dict, target_);
18     for (auto& group : graph->fusion_groups) {
19         // 返回了当前 group 内的所有 Op Nodes
20         groups.push_back(std::move(group->CollectNodes()));
21         // 【【3.1 将 op/group 转为 LoweredFunc】】
22         local_lowered_funcs.emplace_back(std::move(op_lowerer.Lower(group)));
23     }
24 } else {
25     // group 没有分出来 fusion_groups, 此时对 op 挨个处理, 挨个转换成 LoweredFunc
26     // 开启了 cinn_ir_schedule, 则生成 lowered_func 时需要调用
    GetOpFuncWithIRSchedule, 否则调用 GetOpFunc 即可
27     // 【注意】cinn_ir_schedule 开启时, 表示使用新的 schedule 原语【此时应该需要 op
    的 schedule 函数】, 需要 inferdtype 和 infershape 信息
28 }
29 }
30 {
31     utils::RecordEvent("GraphCompiler ProcessFunction",
    utils::EventType::kOrdinary);
32     for (auto&& lowered_func : lowered_funcs) {
33         // 设置 scope 中 tensor 的 shape 和 dtype, 到这里又回到了 scope 语境下
34         this->ProcessFunction(lowered_func);
35     }
36 }
37 // 【【3.2 LoweredFunc 组成 ir::Module, 然后经过低层 pass 优化, 生成源码】】
38 // 【【3.2.1 LoweredFunc 组成 ir::Module】】
39 // 返回类型 backends::Compiler
40 compiler_ = backends::Compiler::Create(target_);
41 // 优化 m_builder_ 里面的 module_, 返回值 ir::Module + (对 Module 的低层 pass)
42 auto build_module = m_builder_.Build();
43 // X86 下的代码生成, 看起来是自己手写的
44 if (this->target_.arch == Target::Arch::X86) {
45     CodeGenCX86 codegen(this->target_, CodeGenCX86::Feature::AVX512);
46     codegen.SetInlineBuiltinCodes(false);
47     // 对 build_module 做编译, 返回 codegen 结果, 为 string, 此处的 out 结果并未用
    到?
48     auto out = codegen.Compile(build_module, CodeGenC::OutputKind::CImpl);
49 }
50 {
51     utils::RecordEvent("GraphCompiler BackendsBuild",
    utils::EventType::kOrdinary);
52     // 【【3.2.2 对ir::Module进一步进行Lower, 直到生成源码, 并生成函数指针】】
53     compiler_->Build(build_module, options.attached_code);
54 }

```

```
55
56 // 【【3.3 遍历 groups 里的 Nodes, 每个 op node 都对应一个 Instruction】】
57 auto instructions = BuildInstructions(groups, options.groups.empty() ?
graph->fusion_groups : options.groups);
58 // scope_>EraseVar(var_name), 从 scope_ 中移除没用到的 variable
59 if (options.remove_unused_variables) { }
60 // 根据 instructions 中变量的生命周期, 为其插入 malloc 和 free 指令
61 if (options.with_buffer_handle_instruction_inserted) { }
62
63 // 在编译期实例化所有变量, 初始化 scope_ 里面的 Tensor, set_buffer
64 if (options.with_instantiate_variables) { }
65 // 【【3.4 返回结果 runtime_program 】】
66 GraphCompiler::CompilationResult result;
67 result.runtime_program.reset(new Program(scope_, std::move(instructions)));
68 return result;
69 }
```

3.1 【核心逻辑】将 op/group 转为 LoweredFunc

op的pattern分为6类，即上层的算子可能有几百个，但是到cinn的基础算子只有100+，然后这100+会被分为6类，再进行融合。同一类算子可以做相同的处理，而不是针对单个算子处理，例如elementwise+relu，elementwise+sigmoid等就是类似的处理。

复杂度	类别	特征	融合能力	算子
1	<u>kElementWise</u>	所有输入、输出形状相同	相互之间可以有条件融合，未来通过Cost Model判定	Unary、Binary等
2	<u>kBroadcast</u>	输入形状是输出形状子集，且位置对应		<u>BroadcastTo</u>
3	<u>kInjective</u>	可以将输出的轴单射到一个输入的轴		<u>Concat</u> 、 <u>Slice</u> 等
4	<u>kReduction</u>	输出形状是输入形状的子集		Reduce
5	<u>kOutFusable</u>	复杂算子	计算结果可以和 <u>kElemWise</u> 算子融合	<u>MatMul</u> 、 <u>Convolution</u>
6	<u>kNonFusable</u>	复杂算子	不能和任何算子融合	Split等

</> cinn/hlir/framework/op_lowering.cc

C++ | 收起 ^

```
1 std::vector<ir::LoweredFunc> OpLowerer::Lower(GroupPtr& group) {
2   group->input_names.clear();
3   group->output_names.clear();
4   if (FLAGS_cinn_ir_schedule) {
5     switch (group->op_pattern_kind) {
6       // 根据当前 group 的 op_pattern_kind 调用不同的 LowerOp 函数
7       case framework::kElementWise:
8       case framework::kBroadcast:
9       case framework::kInjective:
```



```

10      // 【自动调优】IRElementwiseSchedule, schedule是指运算过程的优化和调度, 包括
      vectorize, tile, unroll等
11      return IRLowerOp(&OpLowerer::IRElementwiseCompute,
      &OpLowerer::IRElementwiseSchedule, group);
12      case framework::kReduction:
13          return IRLowerOp(&OpLowerer::IRReduceCompute,
      &OpLowerer::IRReduceSchedule, group);
14      case framework::kOutFusible:
15          LOG(FATAL) << "Group Pattern Kind kOutFusible Is Not Implemented!";
16      case framework::kNonFusible:
17          return IRLowerNonFusibleOp(group, /*apply_impl_schedule = */ true);
18  }
19  }
20  }

```

对 group 做 compute、schedule、pass-低层

</> OpLowerer::IRLowerOp

C++ | 收起 ^

```

1 // 似乎没有用到 IRScheduleFunction schedule?
2 std::vector<ir::LoweredFunc> OpLowerer::IRLowerOp(IRComputeFunction compute,
3                                                    IRScheduleFunction
4                                                    schedule,
5                                                    GroupPtr& group) {
6     poly::StageMap stages;
7     // AST 里面保存了 IRComputeFunction 将 group 转化出 Expr
8     std::vector<Expr> ast_exprs;
9     // 【【3.1.1 do compute. group 没有 fused_sub_groups, 则对整个 group 做计算,
10    &OpLowerer::IRElementwiseCompute】】
11     if (group->fused_sub_groups.size() == 0) {
12         // 每一个 out Tensor 都对应着一个 AST
13         ast_exprs = (this->*compute)(stages, arg_tensors, tensor_map, group,
14                                     group, /*apply_impl_schedule = */ true);
15     } else { }
16     // 【【3.1.2 do schedule.】】
17     ir::ModuleExpr mod_expr(ast_exprs);
18     ir::IRSchedule ir_sch(mod_expr);
19     ir_sch.MergeExprs();
20     IRSchedule(ir_sch, group, tensor_map);
21     // 更新 group->input_names、group->output_names、func_args
22     auto func_body = ir_sch.GetModule().GetExprs().at(0);
23 #ifdef CINN_WITH_CUDA
24     optim::OptimizeExprGPU(&(func_body));
25 #endif

```

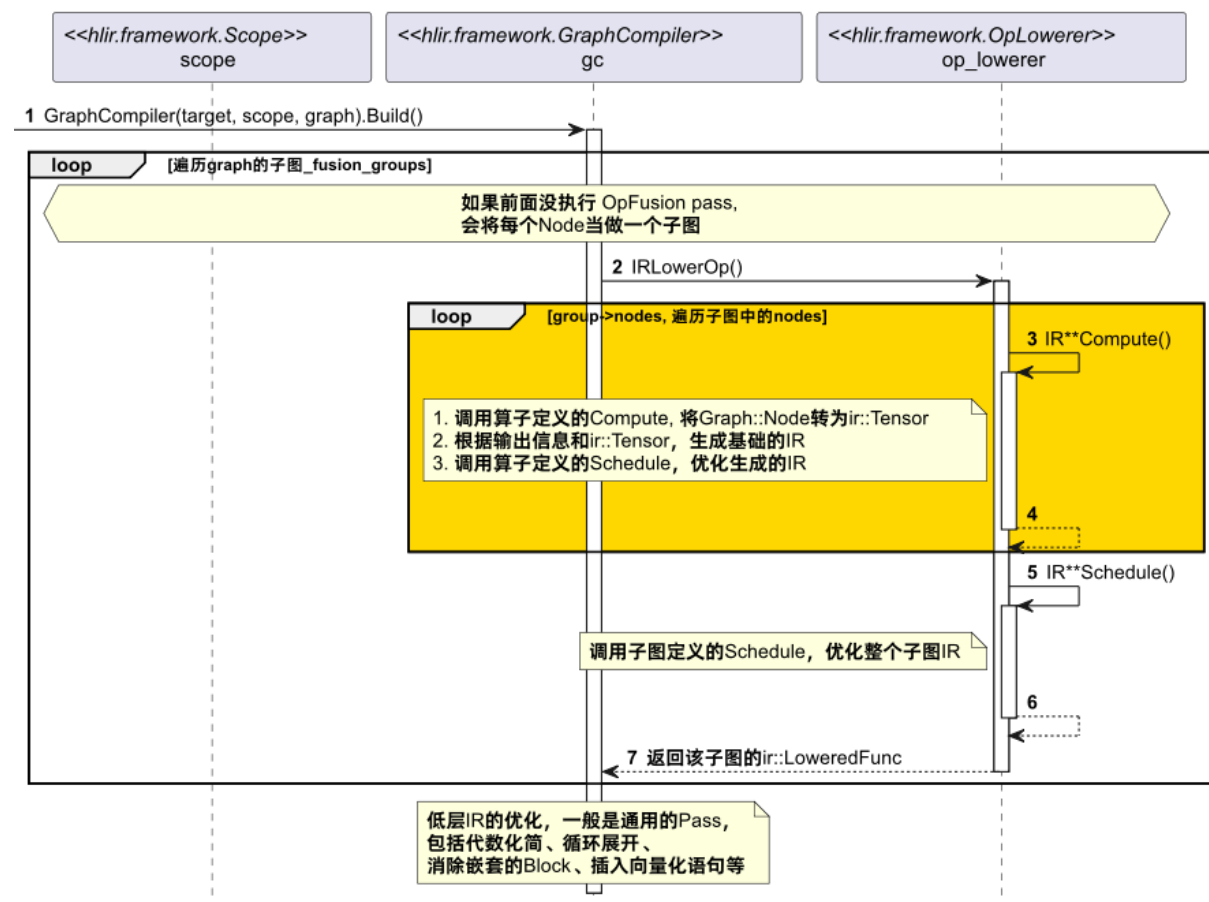


```

23  auto temp_buffers = lang::GetTempBuffers(arg_tensors, stages, func_body);
24
25  // 把当前的 group Make 成一个 _LoweredFunc_
26  auto func = ir::_LoweredFunc::_Make(group->GetFuncName(), func_args,
    ir_sch.GetModule().GetExprs().at(0), temp_buffers);
27  // 【3.1.3 对当前 group 的算子层 IR Expr 做优化, 也是一种低层 Pass? 】
28  func = optim::Optimize(Expr(func), target_, false).as_lowered_func_ref();
29  return {func};
30 }

```

3.1.1 do compute 调用 &OpLowerer::IRElementwiseCompute



对于每一个Op Node, 从注册的StrategyFunction中找出该Op对应的strategy, 然后依次调用 Compute, LowerVec, Schedule。

</> cinn/hlir/framework/op_lowering.cc

C++ | 收起 ^

```

1  std::vector<Expr> OpLowerer::IRElementwiseCompute(poly::StageMap& stages,
2                                                    std::vector<ir::Tensor>&
   func_tensors,
3
   std::unordered_map<std::string, ir::Tensor>& tensor_map,
4                                                    const GroupPtr& group,

```

```

5                                     const GroupPtr& sub_group,
6                                     bool apply_impl_schedule) {
7     // True
8     auto& strategy = Operator::GetAttrs<StrategyFunction>("CINNStrategy");
9
10    std::vector<Expr> ast_exprs;
11    // 遍历 sub_group 中的 op node
12    for (auto& node : sub_group->nodes) {
13        // 将算子 node 的输入输出封装为CINNValue形式
14        // 【【3.1.1.1. do compute, 调用 op 的 compute 函数】】
15        common::CINNValuePack pack = impl-
16        >fcompute(common::CINNValuePack{cinn_inputs});
17
18        // 【【3.1.1.2. lowering 算子】】
19        auto func = lang::LowerVec("fn_" + node->id(), node_stages,
20        tensor_inputs, {}, {}, nullptr, this->target_, true);
21
22        if (apply_impl_schedule) {
23            // 【【3.1.1.3. do ast tree schedule 对 op 进行 schedule】】
24            common::CINNValuePack expr_pack = impl-
25            >fschedule(common::CINNValuePack{schedule_inputs});
26        }
27    }
28 }

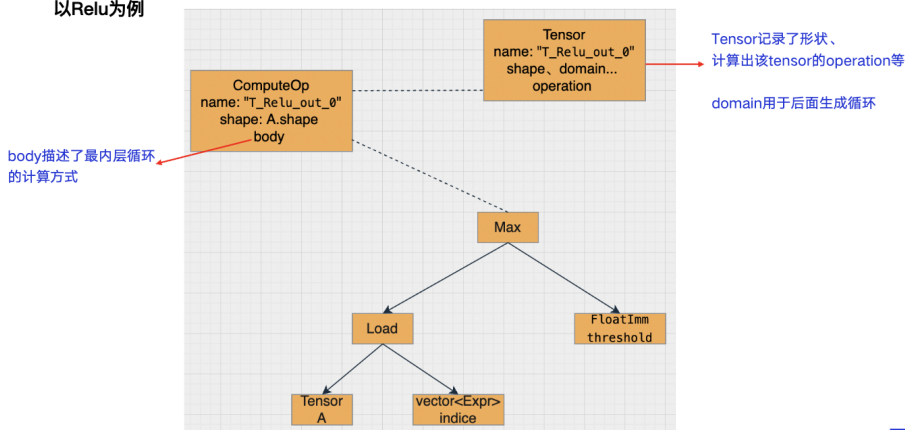
```

3.1.1.1. do compute, 调用 op 的 compute 函数, 以 StrategyForRelu 为例, 组装成 AST Expr

■ CINN IR | Compute



以Relu为例



</> StrategyForRelu: cinn/hlir/op/nn.cc

C++ | 收起 ^

```

1 std::shared_ptr<OpStrategy> StrategyForRelu(const framework::NodeAttr &attrs,
// op 的 attr

```

```

2                                     const std::vector<ir::Tensor>
    &inputs,                          // 输入 Tensor
3                                     const std::vector<Type>
    &out_type,                        // 输出的 dtype
4                                     const
    std::vector<std::vector<int>> &output_shapes, // 输出的 shape
5                                     const Target &target) {
    // 当前的 target
6    // 函数的参数为 Args (可能包含 output_name) 和 RetValue* (输出的指针)
7    framework::CINNCompute relu_compute(=[](lang::Args args, lang::RetValue
    *ret) {
8        // 从 args 里面解析出 input, 然后封装计算逻辑 (此处可能由多个 Expr 组合而成)
9        // ir::Tensor Relu(const ir::Tensor &A, double threshold, const
        std::string &output_name)
10       auto out    = pe::Relu(A.as_tensor_ref(), 0.0, tensor_name);
11       // stages 保存了 Tensor 和 Stage 的映射
12       auto stages = CreateStages({out});
13       *ret        = CINNValuePack{{CINNValue(Expr(out.get()))},
        CINNValue(stages)}};
14    });
15    // 【【3.1.1.3 此处定义了 schedule 】】
16    auto strategy = std::make_shared<framework::OpStrategy>();
17    strategy->AddImpl(relu_compute, GetInjectiveScheduleFunc(output_shapes,
        target), "strategy.relu.x86", 1);
18    return strategy;
19 }

```

3.1.1.2. lowering 算子——生成 LoweredFunc

</> LowerImpl::operator>()()

C++ | 收起 ^

```

1 // 每个 group 对应一个 LoweredFunc
2 std::vector<ir::LoweredFunc> LowerImpl::operator>()() {
3     // 返回 struct Schedule (保存了 graph 内多个 group 的调度信息)
4     // Schedule 里的成员变量
5     // 1. std::vector<ScheduleGroup> groups;--// 从 graph 中切割多个 node 作为一个
        group
6     // 2. std::map<std::string, isl::map> schedule;--// 每个 node 的 isl, 即当前
        node 对应的代码段 (代码生成)
7     auto schedule = poly::CreateSchedule(stages, poly::ScheduleKind::Poly,
        std::vector<std::pair<std::string, std::string>>(deps.begin(), deps.end()));
8     // 返回值为 std::vector<Expr>, 为每个 group 生成一个对应的 expr
9     // group 可以视为一个 ScheduleBlock, 【【往 Block 里面添加需要迭代的变量 i0, i1 和
        axis var】】
10    auto func_body = GenerateFunctionBody(schedule.get());

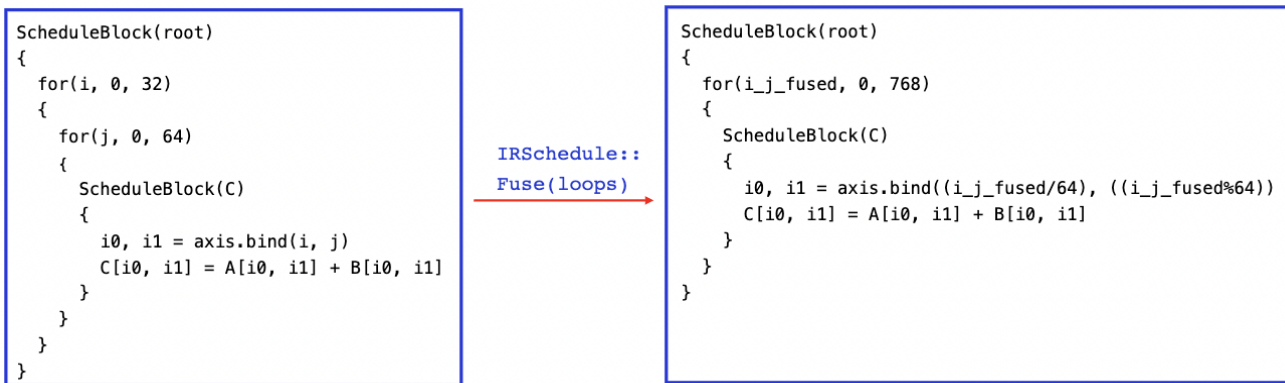
```

```

11
12 // 【注意】已经为每个 group 构造了一个 Expr，接下来该根据 Expr 生成 LoweredFunc
   了!
13 std::vector<ir::LoweredFunc> result;
14 for (auto& func_iterator : func_body) {
15     if (support_ir_schedule_) {
16         func_iterator = ir::ScheduleBlockRealize::Make({},
17 ir::ScheduleBlock::Make({}, {}, {}, common::UniqName("root"),
18 func_iterator));
19     }
20     // 【后面大段代码都是在构造 LoweredFunc 的 Tensor 输入】例如 new_temp_tensors、
   func_temp_tensors
21
22     // 【构造了 LoweredFunc】
23     ir::LoweredFunc func;
24     if (target_ == common::DefaultNVGPUPUTarget()) {
25         auto func_args2 = GenFuncArgForSplitKernel(func_iterator,
26 new_temp_tensors);
27         .....
28         func = ir::_LoweredFunc_::Make(new_fn_name, func_args2, func_iterator,
29 temp_buffers);
30     } else {
31         auto func_args = GenerateFunctionArgumentList(func_iterator);
32         func = ir::_LoweredFunc_::Make(fn_name_, func_args,
33 func_iterator, temp_buffers);
34     }
35
36     if (support_ir_schedule_) {
37         // 对 func->body 做简化
38         optim::TransformPolyForToFor(&func->body);
39         optim::RemoveNestedBlock(&func->body);
40         // 将 func 又封了一层 Block
41         func->body = ir::Block::Make({func->body});
42         result.push_back(ir::LoweredFunc(func.get()));
43     } else { }
44 }
45 return result;
46 }

```

3.1.1.3. do ast tree schedule 对 op 进行 schedule



最后进行Schedule，Schedule就是对计算的调度，像刚才这种2层for循环进行Fuse操作，变成一层，或者进行split变成更多层，以优化cache等等

其他的schedule操作还有compute at，compute inline，把某层 for 循环bind到cuda的某个轴上等等。

Schedule的操作本质上是对AST结构的修改，CINN中是用一系列 Visitor 去访问IR树来实现。

</> cinn/hlir/op/op_util.cc

C++ | 收起 ^

```
1 // 主要是对 for 循环做优化
2 CINNSchedule GetInjectiveScheduleFunc(const std::vector<std::vector<int>>&
   output_shapes,
3                                     const Target& target,
4                                     bool vectorizable) { // True
5   return CINNSchedule( [=](lang::Args args, lang::RetVal* ret) {
6     if (FLAGS_cinn_ir_schedule) {
7       common::CINNValuePack arg_pack = args[0];
8       // 1. 获取 out 的 AST Expr。从 args[0] 中取出所有的 Expr，【疑问】取出来的应该是 out ?
9       std::vector<Expr> vec_ast;
10      for (int i = 0; i < arg_pack.size(); i++) { }
11      ir::ModuleExpr mod_expr(vec_ast);
12      ir::IRSchedule ir_sch(mod_expr);
13      // 清除了 ScheduleBlockRealize, 将 所有 ScheduleBlock 合并到一个
        ScheduleBlockRealize 里面
14      ir_sch.MergeExprs();
15      // 对 for 循环做优化，先提取出 block 中的多层嵌套 for 循环，然后将 for 循环摊
        平，再尝试按照特定的维度给 bind 起来
16      pe::IRInjectiveSchedule(ir_sch, output_shapes.front(), target);
17      std::vector<common::CINNValue>
        res{common::CINNValue(ir_sch.GetModule().GetExprs().at(0))};
18      *ret = common::CINNValuePack{res};
19    } else { }
20  });
21 }
```

3.1.2 do schedule.

</>

C++ | 收起 ^

```
1 // group 层面的四行代码, 与 op 层面做 schedule 的代码非常像 【【略】】
2 ir::ModuleExpr mod_expr(ast_exprs);
3 ir::IRSchedule ir_sch(mod_expr);
4 ir_sch.MergeExprs();
5 IRSchedule(ir_sch, group, tensor_map);
6
```

3.1.3 对当前 group 的算子层 IR Expr 做优化, 像是低层 Expr 的 Pass——实际上对 Expr 做 Optimize

</> Expr Optimize

C++ | 收起 ^

```
1 Expr Optimize(Expr e, Target target, bool runtime_debug_info, bool
  remove_gpu_for_loops) {
2   CHECK(e.defined());
3   auto copied = IRCopy(e);
4
5   FoldCINNCallArguments(&copied);
6   TransformPolyForToFor(&copied);
7   ReplaceConstParamToInteger(&copied);
8   CastSimplify(&copied);
9   Simplify(&copied);
10  UnrollLoop(&copied);
11  VectorizeLoops(&copied, target);
12 #ifdef CINN_WITH_CUDA
13  if (FLAGS_cinn_ir_schedule && copied.as_lowered_func()) {
14    ir::SetCudaAxisInfo(&copied); }
15  if (remove_gpu_for_loops) { RemoveGpuForloopsAxis(&copied); }
16  CudaSyncThreadsDropIfThenElse(&copied);
17 #endif
18  RemoveNestedBlock(&copied);
19  MapExternCall(&copied, target);
20  ExternCallMultiOutputShallowStore(&copied);
21  CastSimplify(&copied);
22  Simplify(&copied);
23  IfSimplify(&copied);
24  if (runtime_debug_info) {
25    InsertDebugLogCallee(&copied);
26  }
```

```

25     }
26     return copied;
27 }

```

其他：另一个被称作低层 Pass 的——是对 `ir::Module` 做 `Optimize`

</> Module Optimize

C++ | 收起 ^

```

1 // 【注意】低层的 Pass 优化
2 // 比如循环展开、向量化、消除嵌套的Block、代数化简等等。
3 // 低层Pass的实现也是继承IRMutator基类，遍历AST树，在Visit函数中用优化后的Node结构替换
  原来的Node。
4 ir::Module Optimize(const ir::Module& module, const Target& target) {
5     auto copied = IRCopy(Expr(module));
6     if (FLAGS_cinn_ir_schedule) {
7         UnrollLoop(&copied);
8         VectorizeLoops(&copied, Target());
9     }
10    RemoveScheduleBlock(&copied);
11    LowerFunctionCallBindVars(&copied);
12    CallArgListToPodValue(&copied);
13    LowerIntrin(&copied, target);
14
15    return copied.as_module_ref();
16 }

```

3.2 LoweredFunc 组成 ir::Module，然后经过低层 pass 优化，生成源码

3.2.1 LoweredFunc 组成 ir::Module，依次处理 LoweredFunc，更新 scope 和 m_builder (ir::Module)

</> GraphCompiler::ProcessFunction

C++ | 收起 ^

```

1 // 设置 scope 中 tensor 的 shape 和 dtype，到这里又回到了 scope 语境下
2 void GraphCompiler::ProcessFunction(const std::vector<ir::LoweredFunc>&
  lowered_funcs) {
3     for (auto&& func : lowered_funcs) {
4         std::vector<std::string> input_args;
5         std::vector<std::string> output_args;
6         for (auto&& arg : func->args) {
7             // 解析 func->args, 更新 input_args 和 output_args, 并更新 scope 中的
              Variable
8         }

```



```

9     function2input_args_[func->name] = input_args;
10    function2output_args_[func->name] = output_args;
11    m_builder_.AddFunction(func);    // 此函数的实现如下
12 }
13 }
14
15 void Module::Builder::AddFunction(ir::LoweredFunc func) {
16     // 此处先对 func 做优化, 再往 module 里面 push_back
17     optim::Simplify(&(func->body));
18     optim::SimplifyForLoops(&(func->body));
19     optim::SimplifyBlocks(&(func->body));
20     func->body = optim::Optimize(func->body, module_->target);
21     module_->functions.push_back(func);
22 }
23
24 // Builder 里面只有一个数据结构: ir::_Module_
25 struct Builder {
26     Builder(const std::string& name, const Target& target) :
27         module_(common::make_shared<ir::_Module_>()) {
28         module_->name = name;
29         module_->target = target;
30     }
31     void AddFunction(ir::LoweredFunc func);
32     void AddBuffer(ir::Buffer buffer);
33     void Clear();
34     Module Build();
35 private:
36     Shared<ir::_Module_> module_;
37 };

```

3.2.2 对 ir::Module 进一步进行 Lower, 对 Module 做低层 pass, 直到生成源码, 并生成函数指针

</>

C++ | 收起 ^

```

1 // 返回类型 backends::Compiler
2 compiler_ = backends::Compiler::Create(target_);
3 // 优化 m_builder_ 里面的 module_, 返回值 ir::Module, 此处用低层的 Pass 对
  ir::Module 做优化。涉及两行代码:
4 // 1. auto res = ir::Module(module_.get());
5 // 2. optim::Optimize(res, module_->target);

```

```

6  auto build_module = m_builder_.Build();
7
8  {
9      // 对 ir::Module 进一步进行 Lower, 直到生成可执行代码。
10     compiler_>Build(build_module, options.attached_code);
11 }

```

</> cinn/backends/compiler.cc

C++ | 收起 ^

```

1 void Compiler::Build(const Module& module, const std::string& code) {
2     if (target_.arch == Target::Arch::NVGPU) {
3         // 更新 cuda_module_ (cuda_module_ 里面保存了 ptx 指针), engine_
        >Link<CodeGenCUDA_Host>(host_module)
4         CompileCudaModule(module, code);
5     } else if (target_.arch == Target::Arch::X86) {
6         // 只需要做 engine_>Link<CodeGenX86>(module)
7         CompileX86Module(module);
8     } else {
9         CINN_NOT_IMPLEMENTED
10    }
11 }
12
13 class Compiler final {
14 private:
15     Target target_;
16     // CompileCudaModule 函数更新了 engine_ 和 cuda_module_
17     std::unique_ptr<ExecutionEngine> engine_;
18 #ifdef CINN_WITH_CUDA
19     std::unique_ptr<runtime::cuda::CUDAModule> cuda_module_;
20 #endif
21 };

```

- 对 device code

- 使用nVRTC::Compiler, 将cuda c代码编译为nv ptx。
 - nVRTC是nv提供的runtime compiler, 详见文档: <https://docs.nvidia.com/cuda/nvrtc/index.html>
 - 主要函数包括: nvrtcCreateProgram, nvrtcCompileProgram, nvrtcGetPTX等, 最后返回ptx string。
 - ptx里面实际上是编译好的函数, PTX就是Parallel Thread eXecution的缩写, 可以打印出来查看, 类似汇编。
 - 可以通过cuModuleLoadDataEx和cuModuleGetFunction得到cuFunction。然后通过cuLaunchKernel(cuFunction...)去运行。

- 对host code

- 使用llvm进行codegen。

</> CompileCudaModule — —cinn/backends/compiler.cc

C++ | 收起 ^

```

1 void Compiler::CompileCudaModule(const Module& module, const std::string&
  code) {
2 #ifdef CINN_WITH_CUDA
3   // 【【3.2.2.1 以 Cuda 为例, 首先把 Module 拆分为 host 端和 device 端】】
4   // host_module_builder 中 op->body 都是 ir:Call
5   // device_module_builder 直接把 module 中的 expr 给 push 进去
6   auto _host_module_device_module_ = SplitCudaAndHostModule(module); //
  NOLINT
7   auto& host_module =
  std::get<0>(_host_module_device_module_);
8   auto& device_module =
  std::get<1>(_host_module_device_module_);
9
10  std::string source_code;
11  if (code.empty()) {
12    // 【【3.2.2.2 实际上调用的是 device_module 的 print 函数】】
13    // 对于device端, 使用CodeGenCuda_dev去遍历AST生成源码
14    CodeGenCUDA_Dev codegen(target_);
15    source_code = codegen.Compile(device_module);
16  } else { source_code = code; }
17  SourceCodePrint::GetInstance()->write(source_code);
18  // 【【3.2.2.3 调用CUDA编译器得到PTX汇编代码, 根据源码生成ptx, 最终的返回结果是
  std::string ptx】】
19  backends::nVRTC::Compiler compiler;
20  auto ptx = compiler(source_code);
21  cuda_module_.reset(new CUDAModule(ptx, compiler.compile_to_cubin() ?
  CUDAModule::Kind::CUBIN : CUDAModule::Kind::PTX));
22
23  // 【【3.2.2.4 注册 CUDA 的 Kernel 的函数符号表, 函数名 fn_xxxx_name_kernel_, 最
  终会注册到 JIT 引擎中】】
24  RuntimeSymbols symbols;
25  for (auto& fn : device_module.functions()) {
26    std::string kernel_fn_name = fn->name;
27    // 返回值是 CUfunction
28    auto fn_kernel = cuda_module_->GetFunction(0,
  kernel_fn_name);
29    symbols.RegisterVar(kernel_fn_name + "_ptr_",
  reinterpret_cast<void*>(fn_kernel));
30  }
31  // 【【3.2.2.5 编译得到 host 端二进制代码, 并注册相应函数符号表, 函数名
  fn_xxxx_name】】

```

```

32  engine_ = ExecutionEngine::Create(ExecutionOptions(), std::move(symbols));
33  // host端把CINN IR转成LLVM IR, 利用LLVM去编译。
34  // 【【3.2.2.6 调用AddIRModule, 把Module注册到LLVM的jit engine中, jit引擎会为
    Module中的所有函数生成可执行的二进制码】】
35  engine_>Link<CodeGenCUDA_Host>(host_module);
36 #else
37  CINN_NOT_IMPLEMENTED
38 #endif
39 }

```

3.3 遍历 groups 里的 Nodes, 每个 op node 都对应一个 Instruction

Instruction 中保存了可执行函数地址——查找函数指针, 从 jit 引擎中找出编译好的可执行代码, 并且绑定函数地址

</> cinn/hlir/framework/graph_compiler.cc

C++ | 收起 ^

```

1 // 遍历 groups 里的 Nodes, 每个 op node 都对应一个 Instruction
2 std::vector<std::unique_ptr<Instruction>> GraphCompiler::BuildInstructions(
3     const std::vector<std::vector<Node*>>& groups, const
4     std::vector<std::shared_ptr<Graph::Group>>& fusion_groups) {
5     std::vector<std::unique_ptr<Instruction>> instructions;
6     auto topo_order = graph->topological_order();
7     auto& nodes     = std::get<0>(topo_order);
8     auto& edges     = std::get<1>(topo_order);
9     for (int idx = 0; idx < groups.size(); ++idx) {
10         auto& group = groups[idx];
11         fusion_group = fusion_groups[idx];
12         // 处理 group 中的唯一一个 node
13         if (group.size() == 1) {
14             auto node = group[0];
15             auto instr_name = node->op()->name;
16             // reshape op 对 buffer 进行 reuse
17             if (node->op()->name == "reshape" &&
18                 compile_options_.with_instantiate_variables) { }
19             // 构造 Instruction
20             auto instr = std::unique_ptr<Instruction>(
21                 new Instruction(target_,
22                     scope_.get(),
23                     fusion_group.get() ? fusion_group->input_names :
24                     OpGetInputNames(node),
25                     fusion_group.get() ? fusion_group->output_names :
26                     OpGetOutputNames(node),

```

```

23         instr_name));
24     // 只处理 NVGPU 场景下的操作, 对于一些 op 做特殊处理, 为 instr 添加相应的属性
25     if (target_.arch == Target::Arch::NVGPU) {
26         if (node->op()->name == "conv2d") { }
27         else if (node->op()->name == "depthwise_conv2d") { }
28         else if (node->op()->name == "pool2d") { }
29         else if (node->op()->name == "softmax") { }
30         else if (node->op()->name == "cublas_gemm" || node->op()->name ==
31             "cublas_matmul") { }
32         }
33         std::string op_func_name = fusion_group.get() ? fusion_group-
34             >GetFuncName() : GetOrGenFullFuncName(GenOpFuncName(node));
35         // 查找函数指针, 从 jit 引擎中找出编译好的可执行代码, 并且绑定函数地址
36         auto* fn_ptr = compiler->Lookup(op_func_name);
37         instr->SetLoweredFunc(reinterpret_cast<void*>(fn_ptr), op_func_name);
38         // 如果 instruction 有多个 kernel, 那么也要 push_back 其他的 kernel
39         SetSubKernels(instr.get(), op_func_name);
40         // 当前 op 需要 pre_run
41         if (node->attrs.attr_store.count("pre_run")) {
42             instr->pre_run = absl::get<bool>(node->attrs.attr_store["pre_run"]);
43         }
44         // 当前 instr 已经确认完毕, instr 不会再修改
45         instr->Finalize();
46         instructions.push_back(std::move(instr));
47     } else { /* 对 group 中的每个 node 做处理*/ }
48 }
49 return instructions;
50 }

```

4. 按顺序执行 Instruction

</> cinn/hlir/framework/graph_compiler.cc

C++ | 收起 ^

```

1 void Program::Execute(const std::map<std::string, cinn_pod_value_t>*
2     name2podargs, void* stream, bool use_cache) {
3     for (auto& ins : instrs_) {
4         ins->Run(name2podargs, false, stream, use_cache);
5     }
6     #ifdef CINN_WITH_CUDA
7     if (instrs_[0]->target_.arch == Target::Arch::NVGPU && stream == nullptr) {
8         CUDA_CALL(cudaDeviceSynchronize());
9     }
10 }

```

```

9 #endif
10 }

```

</> cinn/hlir/framework/instruction.cc

C++ | 收起 ^

```

1 void Instruction::Run(const std::map<std::string, cinn_pod_value_t>*
  name2podargs,
2
3         bool dryrun,
4         void* stream,
5         bool use_cache) {
6     if (function_name_ == "no_run") { return; }
7     {
8         if (!use_cache || args_cached_.size() != size()) {
9             // 从 scope_ 或者 name2podargs 中找出 input 和 output 对应的 Tensor, 取出其
10             Buffer中的cinn_buffer_t*, 与args_cached_关联, 放到 args_cached_ 里面
11             UpdateArgsCache(name2podargs);
12         }
13     }
14 // 对于特殊的 instruction, 执行特化的 函数, 而非代码生成出来的结果
15 #if defined(CINN_WITH_CUDA) && !defined(CINN_WITH_CUDNN)
16     if (function_name_ == "cublas_gemm" && target_.arch == Target::Arch::NVGPU)
17     { }
18     else if (function_name_ == "cublas_matmul" && target_.arch ==
19             Target::Arch::NVGPU) { }
20     else { }
21 #elif defined(CINN_WITH_CUDNN)
22     auto& pod_args = args_cached_[0];
23     // Here conv2d and depthwise_conv2d are implemented by one cudnn api
24     cudnnConvolutionForward
25     if ((function_name_ == "conv2d" || function_name_ == "depthwise_conv2d") &&
26         target_.arch == Target::Arch::NVGPU) { }
27     else if (function_name_ == "pool2d" && target_.arch == Target::Arch::NVGPU)
28     { }
29     else if (function_name_ == "softmax" && target_.arch ==
30             Target::Arch::NVGPU) { }
31     else if (function_name_ == "mul" && target_.arch == Target::Arch::NVGPU) {
32     }
33     else if (function_name_ == "cublas_gemm" && target_.arch ==
34             Target::Arch::NVGPU) { }
35     else if (function_name_ == "cublas_matmul" && target_.arch ==
36             Target::Arch::NVGPU) { }
37     else { }
38 #else
39     // 执行函数指针

```

```

29  for (int idx = 0; idx < fn_ptrs_.size(); ++idx) {
30      auto& pod_args = args_cached_[idx];
31      if (!dryrun) {
32          if (target_ == common::DefaultNVGPUPUTarget()) {
33              ((lower_func_ptr_g)fn_ptrs_[idx])(static_cast<void*>(pod_args.data()),
34              pod_args.size(), stream);
35          } else {
36              ((lower_func_ptr_t)fn_ptrs_[idx])(static_cast<void*>(pod_args.data()),
37              pod_args.size());
38          }
39      }
40      #endif
41      utils::ProfilerRangePop();
42      if (!cinn::runtime::CheckStringFlagFalse(FLAGS_cinn_self_check_accuracy)) {
43          CheckResults(name2podargs, stream);
44      }
45  }

```

Paddle with CINN（跳转）

☰ 代码学习——Pass体系 & PADDLE_WITH_CINN

可优化点（个人视角下的一些疑问）

1. 数据结构的封装性与合理性？

- 封装性：数据结构的封装度明显不足，故意为之还是设计缺陷？

现在的数据结构，内部用了很多 public 变量 && struct 结构体。看起来不像是在写 C++（面向对象），而是在写 C 函数 + 利用 C++ 特性。

——想要找到一个成员变量，函数调用栈较长，并且1. 获取方式不唯一，2. public 直接修改的方式满天飞，耦合性极高。

举例：OpMapperContext 中的 `std::unordered_map<std::string, Variable>* var_map_{nullptr}`;
（传入的是指针，指向 PaddleModelConvertor 变量）

- 合理性：某些数据结构是必要的吗？

举例：

1. CINN 从前端开始要接触两个概念：Program 和 NetBuilder，这两个 class 功能的重复度很高，是往 Program/Builder 里面添加 Instruction (op)。看代码时容易先入为主的接受这两个概念，然而，再深入思考一下，NetBuilder 的概念是必要的吗？两者能否合并成一个，从而减少理解成本？
 - a. Placeholder 和 Variable 的概念是否可以合并？提供了 `operator Variable()` 转换函数，Placeholder 可以自由转换为 Variable
 - b. Program pass 和 Graph pass 为什么要有两套 pass 体系？Program 转为 Graph 之后统一做 pass？
2. Shared 数据结构是必需的吗？不用 Shared 就不能跨平台兼容了吗？

</> Variable、Placeholder、Instruction的定义：cinn/frontend/syntax.h

C++ | 收起 ^

```

1 // 【Variable 的封装: common::Object --> Variable -->
  common::Shared<_Variable_> 】】
2 // 【注意】_Variable_ 只保存了 dtype 和 shape, 没有保存 buffer
3 struct _Variable_ : public common::Object {
4     std::string id;
5     // 数据类型 和 shape
6     common::Type type;
7     std::vector<int> shape;
8     bool is_const = false;
9
10    const char* type_info() const override { return __type_info__; }
11    static constexpr char* __type_info__ = "cinn_frontend_variable";
12 };
13
14 struct Variable : public common::Shared<_Variable_> {
15     explicit Variable(const std::string& id_hint = "") :
16         common::Shared<_Variable_>(common::make_shared<_Variable_>()) {
17         if (!id_hint.empty()) CheckVarNameValid(id_hint);
18         get()->id = id_hint.empty() ? common::Context::Global().NewName("var") :
19         id_hint;
20     }
21
22     void set_id(const std::string& id) { operator->()->id = id; }
23     void set_const(bool is_const) { operator->()->is_const = is_const; }
24     bool is_const() { return operator->()->is_const; }
25
26     _Variable_* operator->() { return get(); }
27     const _Variable_* operator->() const { return get(); }
28 };

```

2. AST 代码生成的设计文档与侵入式开发成本？

AST 以及代码生成模块，应该算是 CINN 的核心内容之一，但是现在如何熟悉其中的设计理念与方案？

学习成本：isl 概念与应用，Stage 概念与应用

数据结构：似乎是过时的，先进性补足

正确性保证：为什么生成代码的精度对不上？（理论上至少应该和组合算子的结果一致？）

扩展性支持：0D-Tensor 扩展需要支持什么？新算子加入需要支持什么？

个人设想

当前对于 CINN 的理解较为浅薄，学习时间大概只有两周左右，后面是一些个人倾向较深的、“不负责任”的脑洞，估计有些地方可能经不起推敲。

短期工作——正确性保障

（基础功能，建立信心，Q级别？）

1. pass 正确性与依赖关系保障：现在前端的 program pass 和 graph pass 有无耦合，能否简化？底层 pass 的正确性？
2. 算子精度对齐保障：生成代码与手写代码的正确性对比
3. 算子功能对齐保障：我们是否排查过 CINN 和 Paddle 算子的对应情况？能否完全对齐？
4. 代码风格统一（外部开发者）：目前代码风格里注释不会自动换行，代码格式与 Paddle 有区别
5. 报错信息优化

中期工作——架构合理性

（几个Q的级别？）

1. 数据结构简化与解耦
2. 代码生成重构（竞品调研 + 核心模块）
3. 常用机制支持：0D-Tensor、动态 shape 等常用机制的支持，易用性提升

长期工作——性能先进性

（开展时间较长，或者某些工作放到后期优化）（通用优化，几个Q的级别？）：

1. 专用 VS 通用的性能优化？短期看专用，长期看通用
2. 自动调优（两个核心问题：部署的低时延要求 + 调优出明显的通用性能优势）（能看到的框架内少数需要工程+research结合的工作，难度极大）
3. 默认后端切换，CINN 作为 paddle 默认后端以后，还有必要作为一个单独的 repo 存在吗？

